CHAPTER 1

Regions, Not Durations

The word *lifetime* suggests a duration: how long something lives, measured on a clock that starts when a value is created and stops when it is destroyed. That picture is close enough to be useful and wrong enough to mislead. It cannot explain why a reference to a vector stops blocking a `push` a few lines before the end of the block, why the same borrow can be fine on one branch of an `if` and an error on the other, or why writing `'static` on a reference to a local variable produces an error instead of a reference that lives forever.

The borrow checker does not think in durations. For the compiler, a lifetime is a **region**: a set of places in the code of a function where a reference must be valid. A region has no length in seconds and no existence at run time. It is computed from the code, checked, and thrown away before the program is built. Once you see lifetimes as regions, most borrow-checker errors read as what they are: two sets of program points that overlap where they must not, or a set that must contain a point it cannot reach.

This chapter builds that model with small programs, each one either compiled and run or rejected by the compiler, and reads the diagnostics with you. Everything else in the book is built on it.

In this chapter you will learn:

- what a lifetime is to the borrow checker: a region of a function body, never a run-time value, erased before code generation;
- the difference between the scope of a value, the scope of a variable that holds a reference, and the region of a borrow;
- how non-lexical lifetimes end a borrow at its last use, path by path, through branches and loops;
- the rule "aliasing XOR mutation" behind errors E0502, E0499 and E0506;
- why lifetime annotations describe relationships and never extend anything, and why the signature is the contract (E0597, "lifetime may not live long enough");
- how to read a borrow-checker diagnostic label by label, and turn it into a fix;
- the lifetime notation used in the rest of the book, including implied bounds.

1.1 A lifetime is not a run-time thing

Start with what a lifetime is not. A reference in a running Rust program is a pointer (plus a length, for a slice or a `str`). There is no field that records how long it may be used, no counter, no check at

the moment you dereference it. All of that work happens at compile time, and its result is a yes or a no: the program is accepted, or it is rejected with a diagnostic.

You can see the erasure from inside a program. The full program is in `code/examples/ch01_erased.rs`:

```
fn first_word(text: &str) -> &str {
    text.split(' ').next().unwrap_or("")
}

fn main() {
    let literal: &'static str = "regions not durations";
    let owned = String::from("borrowed from a local");
    let local: &str = &owned;

    // One function, one piece of machine code, both kinds of borrow.
    println!("{} / {}", first_word(literal), first_word(local));

    // The name of the type that code generation sees has no lifetime in it...
    let name = type_name::<&'static str>();
    println!("type_name::<&'static str>() = {name}");
    // ...and the two kinds of reference have the same size and layout.
    assert_eq!(size_of::<&'static str>(), size_of::<&str>());
}
```

```
regions / borrowed
type_name::<&'static str>() = &str
```

`first_word` is called once with a string literal, of type `&'static str`, and once with a borrow of a local `String`, whose region ends with `main`. The compiler generates one piece of machine code for `first_word` and uses it for both calls: nothing in the generated code depends on the lifetime. The two kinds of reference have the same size and layout (the `assert_eq!` passes), and when we ask for the name of the type `&'static str`, the compiler answers `&str`: by the time code is generated, the `'static` is gone. (The documentation of `std::any::type_name` warns that its exact text is a best-effort description that may change between compiler versions, so take that line as an illustration rather than a guarantee. The point does not depend on it.)

Three consequences follow, and they shape the rest of the book:

1. **A lifetime cannot be inspected or tested at run time.** There is no `if 'a == 'static`. Anything that needs to know about lifetimes while the program runs must be expressed in types instead. This is why `std::any::TypeId` works only for `'static` types (Chapter 7).
2. **Lifetimes cost nothing.** A program whose lifetimes are written out in full and the same program with every lifetime elided compile to the same code.
3. **The borrow checker must be conservative.** Since nothing is checked while the program runs, the compiler rejects every program it cannot prove safe, including some that would in fact run correctly. Chapter 5 shows the best-known example and the work under way to accept it. When you need checks that depend on run-time behavior, you move them to run time explicitly, with types such as `RefCell` (Chapter 12).

1.2 Regions: sets of program points

If a lifetime is not a duration, what is it? The model that rustc's borrow checker implements comes from RFC 2094, which introduced non-lexical lifetimes and states it in one line: "we will consider lifetimes as a set of points in the control-flow graph" (https://rust-lang.github.io/rfcs/2094-nll.html).

The **control-flow graph** of a function is its body seen as a graph: each statement is a point, and an edge goes from one point to every point that can run next. A straight sequence of statements is a chain; an `if` splits the chain in two and joins it again; a loop adds an edge back to its start. A **region** is a set of points of that graph, and a reference of type `&'a T` comes with a promise:

> At every point in the region `'a`, the value the reference points to is still there (not dropped, not moved), and nobody accesses it in a way that conflicts with the reference.

What "conflicts" means depends on the kind of reference, and we will come back to it below. For now, notice the shape of the promise: it is a statement about a set of points, not about an interval of time. Rustc actually computes regions on a simplified form of the function body (called MIR, for mid-level intermediate representation) whose points are finer than source lines. In this book we reason with lines and statements, which is precise enough for almost every error you will meet; Chapter 5 shows one of the few places where the finer points matter (two-phase borrows).

Three different extents

Three things in a program are easy to confuse because all three are informally called "how long something lives". This program, in `code/examples/ch01_scope_vs_region.rs`, compiles:

```
fn main() {
    let config = String::from("port=8080"); // `config` comes into scope
    let key = &config[..4]; // the borrow is created
    println!("key: {key}"); // last use of `key`: the borrow's region ends here

    let moved = config; // fine: no borrow of `config` is live any more
    println!("moved: {moved}");
} // `key` and `moved` go out of scope here
```

```
key: port
moved: port=8080
```

Look at the three extents involved:

What	From	To
the value `config`	`let config = ...`	`let moved = config` (moved out)
the variable `key`	`let key = ...`	the closing brace of `main`
the borrow `&config[..4]`	the borrow expression	the `println!`, its last use

The **scope of a value** is where it exists: from its creation until it is dropped or moved away. The **scope of a variable** is the block it is declared in, a purely syntactic notion. The **region of a borrow** is where the reference created by the borrow may still be used. Only the third is a lifetime in the borrow

checker's sense, and it is the smallest of the three. The variable `key` is still in scope when `config` is moved, but the borrow it holds is dead, because nothing uses `key` again. So moving `config` is fine.

This is also why "the lifetime of a reference" and "the lifetime of the value it points to" are different things. The region of a borrow must fit inside the scope of the value it borrows, since a reference must never outlast its referent. But it is usually much smaller, and the compiler keeps it as small as the uses of the reference allow. If you used `key` after the move, the region would have to reach past the point where `config` is moved away, which is impossible. That error (E0505, "cannot move out of `config` because it is borrowed") is covered with the other move-related errors in Chapter 5.

1.3 Non-lexical lifetimes: a borrow ends at its last use

The example that made non-lexical lifetimes famous is a vector, a reference to its first element, and a `push`. This version compiles (`code/examples/ch01_nll.rs`):

```
fn main() {
    let mut v = vec![1, 2, 3];
    let first = &v[0];
    println!("{first}"); // last use of `first`: the borrow ends here
    v.push(4); // so this mutable borrow does not overlap it
    println!("{v:?}");
}
```

```
1
[1, 2, 3, 4]
```

The borrow `&v[0]` is created on the second line and used for the last time by the first `println!`. Its region is those two points and nothing more. The `push` on the next line needs a mutable borrow of `v`, which would conflict with a live shared borrow; but the shared borrow is no longer live, so there is no conflict.

#	statement	lexical scope of first (old rule)	NLL region of the borrow &v[0]	access to v
1	let mut v = vec![1, 2, 3];			
2	let first = &v[0];	live	created	shared
3	println!("{first}");	live	last use	
4	v.push(4);	overlap: rejected	no overlap: accepted	mutable
5	println!("{v:?}");	live		shared

The borrow `&v[0]` is live only until its last use, so `v.push(4)` does not overlap it; a rule based on the lexical scope of `first` would keep the borrow alive to the end of the block and reject the program.

The figure compares the region with the scope of the variable `first`. Until the 2018 edition, Rust used the scope: a borrow stored in a variable was live until the end of the block that declared the variable, and this program was rejected. Non-lexical lifetimes, NLL for short, shipped in Rust 1.31 together with the 2018 edition (https://blog.rust-lang.org/2018/12/06/Rust-1.31-and-rust-2018/), and the borrow checker of every edition works this way today. "Non-lexical" means exactly that the region is computed from the uses of the reference, not from the blocks of the source text.

Move the last use of `first` after the `push`, and the region must stretch across it. The compiler rejects this version (`code/compile_fail/ch01_nll_conflict.rs`):

```
pub fn caller() {
    let mut v = vec![1, 2, 3];
    let first = &v[0];
    v.push(4); // needs `&mut v` while `first` is still live
    println!("{first}"); // this later use keeps the shared borrow alive
}
```

```
error[E0502]: cannot borrow `v` as mutable because it is also borrowed as immutable
  --> compile_fail/ch01_nll_conflict.rs:9:5
   |
 8 |     let first = &v[0];
   |                  - immutable borrow occurs here
 9 |     v.push(4); // needs `&mut v` while `first` is still live
   |     ^^^^^^^^^ mutable borrow occurs here
10 |     println!("{first}"); // this later use keeps the shared borrow alive
   |                ----- immutable borrow later used here
```

Read the diagnostic from top to bottom, as a short story told in three labels:

- `immutable borrow occurs here`, under `v` on line 8: the shared borrow starts here. Its region begins at this point.
- `mutable borrow occurs here`, under the whole `v.push(4)` on line 9: this is the conflicting access, and the primary label of the error (the one underlined with `^^^`). `push` takes `&mut self`, so the call borrows `v` mutably.
- `immutable borrow later used here`, under `first` on line 10: the reason the shared borrow is still live on line 9. Because `first` is used here, the region of the shared borrow must contain every point between line 8 and line 10, including the `push`.

The error is not a quirk of the checker. `push` may need more room than the vector has, in which case it allocates a new buffer, moves the elements there and frees the old one. `first` would then point into freed memory, and the `println!` would read it. The borrow checker cannot know whether this particular `push` reallocates, so it rejects every program in which a reference into the vector is live across a call that may.

Liveness follows the value, not the variable

"Last use" needs one refinement. A borrow is live at a point if the reference it produced, *or any reference derived from it*, may be used later. Copying the reference into another variable or storing it in a data structure moves the liveness along with it. In this program (`code/compile_fail/ch01_nll_derived.rs`), the variable `first` is never used after the line that builds `shortlist`:

```
pub fn caller() {
    let mut words = vec![String::from("regions")];
    let first = &words[0]; // last use of the variable `first`...
    let shortlist = vec![first]; // ...but the borrow now lives in `shortlist`
    words.push(String::from("durations"));
    println!("{shortlist:?}");
}
```

```
error[E0502]: cannot borrow `words` as mutable because it is also borrowed as immutable
  --> compile_fail/ch01_nll_derived.rs:10:5
   |
 8 |     let first = &words[0]; // last use of the variable `first`...
   |                  ----- immutable borrow occurs here
 9 |     let shortlist = vec![first]; // ...but the borrow now lives in `shortlist`
10 |     words.push(String::from("durations"));
   |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ mutable borrow occurs here
11 |     println!("{shortlist:?}");
   |                ---------- immutable borrow later used here
```

The compiler still reports the conflict, and its third label points at the use of `shortlist`, not of `first`. The type of `shortlist` is `Vec<&String>`, and the reference inside it is the very borrow created on line 8. In the language of regions, storing `first` into the vector creates a constraint: the region of the borrow must contain every point where the reference in `shortlist` may be used.

How the compiler finds a region

You never write the regions of borrows inside a function body; the compiler infers them. A simplified description of what it does is enough to predict its answers:

1. **Liveness.** For each reference, it finds the points where the reference (or a copy of it) may still be used later, along some path of the control-flow graph. The region of the reference must contain all of them.
2. **Flow.** When a reference is copied, assigned, stored in a structure or passed to a function, the region of the source must contain the region of the destination. This is how the use of `shortlist` above extends the region of the borrow made on line 8.
3. **Smallest solution.** It computes the smallest regions that satisfy all these constraints.
4. **Check.** It walks the function body and, at every access to a variable (a read, a write, a move, a borrow, a drop at the end of a scope), checks that no live borrow conflicts with it. Every conflict is an error.

The borrow conflicts in this chapter are all reported in step 4, but their cause is always in steps 1 and 2: some later use, direct or derived, made a region bigger than the program allows. (The other kind of error you will meet below, "lifetime may not live long enough", comes from step 3: the smallest solution would need a relation between two lifetimes that nothing provides.)

1.4 Shared and mutable borrows: aliasing XOR mutation

Step 4 needs a definition of "conflict". Rust's rule is short: at every point of the program, a value may have **any number of shared borrows, or exactly one mutable borrow**, but not both. It is often summarized as *aliasing XOR mutation*: a value can be reachable through several paths (aliased), or it

can be changed (mutated), but never both at the same point. The owner of the value counts as one of the paths: while a mutable borrow is live, even the variable that owns the value cannot be used.

Since regions are sets of points, the rule allows a lot. This program (`code/examples/ch01_aliasing.rs`) compiles, even though it creates two mutable borrows of the same string:

```
fn main() {
    let mut log = String::from("== report ==");

    // Any number of shared borrows may be live at the same time.
    let whole = &log;
    let title = &log[3..9];
    println!("{whole} / {title}");

    // One mutable borrow at a time: each one ends before the next begins.
    let header = &mut log;
    header.push_str("\nstatus: ");
    let body = &mut log;
    body.push_str("all good");

    println!("{log}");
}
```

```
== report == / report
== report ==
status: all good
```

`whole` and `title` are two shared borrows, live at the same time, which is always allowed. `header` and `body` are two mutable borrows, but `header` is last used before `body` is created, so their regions do not overlap. Finally `println!("{log}")` reads `log` after both have ended.

In `code/compile_fail/ch01_two_mutable.rs` the two mutable regions overlap, and the compiler rejects the program with E0499:

```
pub fn caller() {
    let mut log = String::new();
    let header = &mut log;
    let body = &mut log; // `header` is still live: it is used below
    header.push_str("== report ==\n");
    body.push_str("all good\n");
}
```

```
error[E0499]: cannot borrow `log` as mutable more than once at a time
 --> compile_fail/ch01_two_mutable.rs:8:16
  |
7 |     let header = &mut log;
  |                  -------- first mutable borrow occurs here
8 |     let body = &mut log; // `header` is still live: it is used below
  |                ^^^^^^^^ second mutable borrow occurs here
9 |     header.push_str("== report ==\n");
  |     ------ first borrow later used here
```

The labels follow the same pattern as before: where the first borrow starts, where the conflicting second borrow happens (the primary label), and the later use that keeps the first borrow live (`header.push_str` on line 9).

The rule is symmetric. While a mutable borrow is live, the value cannot even be read through its owner (`code/compile_fail/ch01_mutable_then_shared.rs`):

```rust
pub fn caller() {
    let mut log = String::from("start");
    let writer = &mut log;
    println!("{log}"); // reading `log` needs a shared borrow
    writer.push_str(" end");
}
```

```
error[E0502]: cannot borrow `log` as immutable because it is also borrowed as mutable
  --> compile_fail/ch01_mutable_then_shared.rs:9:16
   |
 8 |     let writer = &mut log;
   |                  -------- mutable borrow occurs here
 9 |     println!("{log}"); // reading `log` needs a shared borrow
   |                ^^^ immutable borrow occurs here
10 |     writer.push_str(" end");
   |     ------ mutable borrow later used here
```

Reading `log` creates a shared borrow (the `println!` macro borrows its arguments), and the mutable borrow held by `writer` is live, because `writer` is used on the next line. The error code is again E0502; only the order of the two borrows changed, and the headline says so: "cannot borrow `log` as *immutable* because it is also borrowed as *mutable*".

Borrowing is not the only access that can conflict. Assigning a new value to a variable drops the old value, so it conflicts with any live borrow of it (`code/compile_fail/ch01_assign_borrowed.rs`):

```rust
pub fn caller() {
    let mut title = String::from("draft");
    let shown = &title;
    title = String::from("final"); // would drop the string `shown` points to
    println!("{shown}");
}
```

```
error[E0506]: cannot assign to `title` because it is borrowed
  --> compile_fail/ch01_assign_borrowed.rs:9:5
   |
 8 |     let shown = &title;
   |                 ------ `title` is borrowed here
 9 |     title = String::from("final"); // would drop the string `shown` points to
   |     ^^^^^ `title` is assigned to here but it was already borrowed
10 |     println!("{shown}");
   |                ----- borrow later used here
```

If the assignment were allowed, the old string `"draft"` would be freed while `shown` still pointed to it. Here is the full set of conflicts this chapter shows, with the code rustc uses for each:

A live borrow…	…conflicts with	Code
shared (`&v`)	a mutable borrow of `v`	E0502
mutable (`&mut v`)	a shared borrow of `v` , including a read	E0502
mutable (`&mut v`)	a second mutable borrow of `v`	E0499
shared or mutable	an assignment to `v`	E0506
shared or mutable	a move or drop of `v`	E0505, E0597

A shared borrow never conflicts with another shared borrow. The last row is about the value disappearing while borrowed: E0597 appears later in this chapter, and E0505 in Chapter 5, together

with two refinements of the rule that make common code compile (two-phase borrows, which accept `v.push(v.len())`, and reborrows).

1.5 Lifetimes in branches and loops

Because a region is a set of points in a graph, and not an interval of lines, a borrow can be live on one path through a function and dead on another. This function (`code/examples/ch01_branches.rs`) borrows the first word, mutates the vector on one branch of an `if`, and uses the borrow on the other:

```
fn describe_or_extend(words: &mut Vec<String>) {
    let first = &words[0];
    if first.len() <= 5 {
        words.push(String::from("durations")); // `first` is dead on this path...
    } else {
        println!("first word is long: {first}"); // ...and used on this one
    }
}

fn main() {
    let mut short = vec![String::from("not")];
    let mut long = vec![String::from("regions")];
    describe_or_extend(&mut short);
    describe_or_extend(&mut long);
    println!("{short:?} {long:?}");
}
```

```
first word is long: regions
["not", "durations"] ["regions"]
```

The condition uses `first`, so the borrow is live there. On the `then` branch, nothing uses `first` again: from the start of that branch, no path leads to a use. On the `else` branch, `first` is printed, so the borrow is live there. The region of the borrow therefore contains the condition and the `else` branch, and not a single point of the `then` branch, where `push` is free to borrow `words` mutably. A rule based on intervals of source text, "from the borrow to its last use", would cover every line from `let first` down to the `println!`, including the `push` written between them, and would reject this function.

Add one use of `first` after the `if`, and the borrow becomes live on both branches, because both lead to that use (`code/compile_fail/ch01_branch_conflict.rs`):

```
pub fn describe_or_extend(words: &mut Vec<String>, verbose: bool) {
    let first = &words[0];
    if verbose {
        words.push(String::from("durations"));
    }
    println!("first word: {first}");
}
```

```
error[E0502]: cannot borrow `*words` as mutable because it is also borrowed as immutable
  --> compile_fail/ch01_branch_conflict.rs:9:9
   |
 7 |     let first = &words[0];
   |                 ----- immutable borrow occurs here
 8 |     if verbose {
 9 |         words.push(String::from("durations"));
   |         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ mutable borrow occurs here
10 |     }
```

```
11 |     println!("first word: {first}");
   |                            ----- immutable borrow later used here
```

The diagnostic names `*words`, the vector that `words` points to, because `words` is itself a reference (`&mut Vec<String>`): the borrows are of the vector behind it. The rest reads as before.

Loops add a back edge to the graph, and with it the possibility that a borrow made in one iteration is live in the next. This program (`code/examples/ch01_loop.rs`) keeps a reference to the word appended in the previous iteration, and compiles:

```
fn main() {
    let mut transcript = String::new();
    let mut previous: &str = "(start)";
    for word in ["regions", "not", "durations"] {
        println!("{previous} -> {word}"); // last use of the previous borrow
        if !transcript.is_empty() {
            transcript.push(' '); // `previous` is dead here: it is overwritten below
        }
        let start = transcript.len();
        transcript.push_str(word);
        previous = &transcript[start..]; // a new borrow of `transcript`
    }
    println!("last word: {previous}; transcript: {transcript}");
}
```

```
(start) -> regions
regions -> not
not -> durations
last word: durations; transcript: regions not durations
```

Follow the borrow `&transcript[start..]` made at the end of the first iteration. Through `previous`, it is live at the start of the second iteration, where the `println!` uses it. After that use, it is dead: the next thing that happens to `previous` is that a new borrow overwrites it, so its old value can never be read again. The region of the first borrow therefore covers the end of the first iteration and the beginning of the second, up to the `println!`, and does not contain the `push` and `push_str` that come after. Each iteration's mutation of `transcript` falls in the gap between the last use of one borrow and the creation of the next. The borrow made in the last iteration is different only in where its uses are: it stays live after the loop ends, because the final `println!` uses `previous`.

Now move the `println!` after the `push_str`, and the previous borrow must survive the mutation (`code/compile_fail/ch01_loop_conflict.rs`):

```
pub fn caller() {
    let mut transcript = String::new();
    let mut previous: &str = "(start)";
    for word in ["regions", "not", "durations"] {
        let start = transcript.len();
        transcript.push_str(word);
        println!("{previous} -> {word}"); // uses the borrow made last time round
        previous = &transcript[start..];
    }
}
```

```
error[E0502]: cannot borrow `transcript` as mutable because it is also borrowed as immutable
  --> compile_fail/ch01_loop_conflict.rs:11:9
   |
11 |         transcript.push_str(word);
```

```
   |         ^^^^^^^^^^^^^^^^^^^^^^^^^ mutable borrow occurs here
12 |         println!("{previous} -> {word}"); // uses the borrow made last time round
   |                   -------- immutable borrow later used here
13 |         previous = &transcript[start..];
   |                     ---------- immutable borrow occurs here
```

This diagnostic looks strange at first, because its labels are out of source order: the shared borrow "occurs" on line 13, *after* the mutable borrow on line 11 that it conflicts with. Read it with the loop in mind. The shared borrow made on line 13 in one iteration is still live when control comes back around to line 11 in the next iteration, because line 12 uses it after that. Whenever the label that starts a borrow appears below the conflict, look for a loop.

There is one pattern involving branches that NLL, as implemented in stable rustc, still rejects even though it is safe: returning a borrow from a function on one path and mutating the borrowed value on another. Chapter 5 covers it as *NLL problem case #3*, together with Polonius, the borrow checker being developed to accept it.

1.6 Annotations describe; they never extend

So far every region was inferred. Lifetime *names*, such as `'a`, appear where the compiler cannot infer anything because it is looking at one function at a time: in signatures and in type definitions. The classic example is a function that returns one of two string slices (`code/examples/ch01_longest.rs`):

```rust
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() >= y.len() { x } else { y }
}
```

The signature declares one lifetime parameter, `'a`, and uses it three times. Read it as a contract between `longest` and its callers:

- the caller picks a region for `'a`, at each call;
- the caller must pass two references that are both valid throughout `'a`;
- in return, it gets a reference that is valid throughout `'a`, and no longer.

Since both arguments must be valid for all of `'a`, the region that a caller can pick is at most the part of the program where *both* are valid: the result is usable only where both inputs are. Nothing here makes either string live longer. The annotation describes a relationship between the three references, and the compiler checks that every call respects it.

A call that uses the result while both inputs are alive compiles. Here is the first of two such callers in the same file:

```rust
fn use_inside() {
    let outer = String::from("a longer string");
    let result;
    {
        let inner = String::from("short");
        result = longest(outer.as_str(), inner.as_str());
        println!("{result}"); // fine: `inner` is still alive here
    }
}
```

```
a longer string
a longer string
```

(The output has two lines because `main` also runs a second caller, shown below.) Move the `println!` after the inner block, and the compiler rejects the call (`code/compile_fail/ch01_longest_dangling.rs`):

```rust
pub fn caller() {
    let outer = String::from("a longer string");
    let result;
    {
        let inner = String::from("short");
        result = longest(outer.as_str(), inner.as_str());
    } // `inner` is dropped here...
    println!("{result}"); // ...but `result` may point into it
}
```

```
error[E0597]: `inner` does not live long enough
  --> compile_fail/ch01_longest_dangling.rs:15:42
   |
14 |         let inner = String::from("short");
   |             ----- binding `inner` declared here
15 |         result = longest(outer.as_str(), inner.as_str());
   |                                           ^^^^^ borrowed value does not live long enough
16 |     } // `inner` is dropped here...
   |     - `inner` dropped here while still borrowed
17 |     println!("{result}"); // ...but `result` may point into it
   |                ------ borrow later used here
```

Write down the constraints and the contradiction appears:

Constraint	Reason
`'a` contains line 17	`result` has type `&'a str` and is used on line 17
the borrow of `inner` lasts for all of `'a`	`inner.as_str()` is passed as a `&'a str`
`inner` is dropped on line 16	the end of its block

The borrow of `inner` would have to be live on line 17, after `inner` has been dropped. No choice of `'a` satisfies all three rows, so the compiler reports E0597, " `inner` does not live long enough". Notice what the compiler did *not* do: it did not look inside `longest` to see which string is returned (for these inputs it would be `outer` , and the program would happen to be safe). It checked the call against the signature, and the signature says that the result may borrow from either argument.

An annotation can only add requirements

Since annotations describe, writing a longer lifetime cannot make a reference last longer. It can only add a requirement that the program then fails to meet (`code/compile_fail/ch01_annotation_static.rs`):

```rust
pub fn caller() {
    let name = String::from("config.toml");
    let forever: &'static str = &name; // asks for a borrow that never ends
    println!("{forever}");
}
```

```
error[E0597]: `name` does not live long enough
  --> compile_fail/ch01_annotation_static.rs:8:33
   |
 7 |     let name = String::from("config.toml");
```

```
   |         ---- binding `name` declared here
 8 |     let forever: &'static str = &name; // asks for a borrow that never ends
   |                  ------------   ^^^^^ borrowed value does not live long enough
   |                  |
   |                  type annotation requires that `name` is borrowed for `'static`
 9 |     println!("{forever}");
10 | }
   | - `name` dropped here while still borrowed
```

The annotation `&'static str` says that the borrow must be valid for the region `'static`, which contains every point of the program, including every point after `name` is dropped at the end of `caller`. The diagnostic adds a kind of label you will see often, under the annotation: the type annotation "requires that `name` is borrowed for `'static`". Such a label tells you *where a requirement came from*, here the type written after `forever:`. Remove the annotation and the program compiles, with a region that ends at the `println!`. The general lesson: when a lifetime error appears, adding or lengthening an annotation never makes a value live longer. The fix is always to change the program (where values live, where references are used, or who owns the data), or to correct a signature that promises more, or demands more, than it should.

The signature is the contract

The body of a function is checked against its signature, and every caller is checked against the signature *and nothing else*. This is the thread that runs through the whole book, and the two halves of it are worth seeing separately.

Callers see only the signature. This function returns its first argument and merely prints the second (`code/compile_fail/ch01_signature_contract.rs`):

```rust
fn first<'a>(x: &'a str, y: &'a str) -> &'a str {
    println!("ignoring {y:?}");
    x
}
```

The result never borrows from `y`. But the signature, which uses `'a` for `y` as well, says that it may, and the same caller as before is rejected with the same error:

```rust
pub fn caller() {
    let outer = String::from("a longer string");
    let result;
    {
        let inner = String::from("short");
        result = first(outer.as_str(), inner.as_str());
    }
    println!("{result}");
}
```

```
error[E0597]: `inner` does not live long enough
  --> compile_fail/ch01_signature_contract.rs:18:40
   |
17 |         let inner = String::from("short");
   |             ----- binding `inner` declared here
18 |         result = first(outer.as_str(), inner.as_str());
   |                                        ^^^^^ borrowed value does not live long enough
19 |     }
   |     - `inner` dropped here while still borrowed
```

```
20 |     println!("{result}");
   |                ------ borrow later used here
```

The fix is in the signature, not in the caller: give `y` its own lifetime, so that the contract says what the body does (`code/examples/ch01_signature_fixed.rs`):

```
fn first<'a, 'b>(x: &'a str, y: &'b str) -> &'a str {
    println!("ignoring {y:?}");
    x
}
```

```
ignoring "short"
a longer string
```

With the same caller, this version compiles: the result has lifetime `'a`, and nothing ties `'a` to the borrow of `inner`.

The body must keep the promise. The other half: the compiler checks the body against the signature, without looking at any caller. Here `longest` is given two independent lifetimes, and the result is declared to borrow from `x` only (`code/compile_fail/ch01_body_vs_signature.rs`):

```
fn longest<'a, 'b>(x: &'a str, y: &'b str) -> &'a str {
    if x.len() >= y.len() { x } else { y }
}
```

```
error: lifetime may not live long enough
 --> compile_fail/ch01_body_vs_signature.rs:7:40
  |
6 | fn longest<'a, 'b>(x: &'a str, y: &'b str) -> &'a str {
  |            --  -- lifetime `'b` defined here
  |            |
  |            lifetime `'a` defined here
7 |     if x.len() >= y.len() { x } else { y }
  |                                        ^ function was supposed to return data with
lifetime `'a` but it is returning data with lifetime `'b`
  |
  = help: consider adding the following bound: `'b: 'a`
```

This error has no code, only the message "lifetime may not live long enough", which the compiler uses whenever a body breaks the lifetime promises of its signature. The labels name the two lifetime parameters, and the primary label explains the broken promise: the function "was supposed to return data with lifetime `'a` but it is returning data with lifetime `'b`". Inside `longest`, `'a` and `'b` are regions chosen by some caller the function cannot see; nothing says that `'b` contains all of `'a`, so returning `y` as a `&'a str` is not justified. The `help` line proposes to add that relation to the contract, as the bound `'b: 'a` (read "`'b` outlives `'a`"; the notation is explained below). Adding it makes the body correct, and moves the requirement to the callers.

Why design the language this way, when the compiler could look at the body of `first` and see that `y` is never returned? Because checking functions one at a time is what makes the result predictable and stable. A caller does not break when the body of a function it calls changes, as long as the signature stays the same; a library can change its implementation without breaking its users; and you, reading a signature, know everything the borrow checker will assume about the function. Every

default the compiler writes for you, from elided lifetimes (Chapter 3) to the hidden bounds of trait objects (Chapter 8), matters because it becomes part of this contract.

1.7 Reading a borrow-checker diagnostic

The diagnostics in this chapter all have the same anatomy, and learning to read it pays off for the rest of the book. Take the error of the dangling caller again:

```
error[E0597]: `inner` does not live long enough
  --> compile_fail/ch01_longest_dangling.rs:15:42
   |
14 |         let inner = String::from("short");
   |             ----- binding `inner` declared here
15 |         result = longest(outer.as_str(), inner.as_str());
   |                                           ^^^^^ borrowed value does not live long enough
16 |     } // `inner` is dropped here...
   |     - `inner` dropped here while still borrowed
17 |     println!("{result}"); // ...but `result` may point into it
   |                 ------ borrow later used here
```

- **The headline** gives the error code and a one-line summary: E0597, "`inner` does not live long enough". The code is the key to the long explanation that `rustc --explain E0597` prints, and to Appendix B of this book.
- **The location** (`-->`) is the file, line and column of the primary label.
- **The primary label**, underlined with `^^^`, marks the operation that the compiler refuses, here the borrow of `inner`: "borrowed value does not live long enough".
- **The secondary labels**, underlined with `---`, are the evidence. They are the points of the regions involved: where the borrowed value was declared ("binding `inner` declared here"), where it stops existing ("`inner` dropped here while still borrowed"), and the use that requires the borrow to be live at that point ("borrow later used here").
- **Notes and help** (`= note:`, `= help:`), when present, explain where a requirement comes from or suggest a change. Read them, but judge the suggestion against what you mean: a suggested bound such as `'b: 'a` makes the body compile by asking more of every caller.

The secondary labels, read in program order, form a timeline. A good habit is to turn every borrow-checker error into four questions:

1. **Where is the borrow created?** ("borrowed value", "immutable borrow occurs here", "first mutable borrow occurs here")
2. **What ends it, or conflicts with it?** ("dropped here while still borrowed", "mutable borrow occurs here", "assigned to here")
3. **Which later use keeps it alive?** ("borrow later used here", "immutable borrow later used here", "first borrow later used here")
4. **Where does the requirement come from?** ("type annotation requires that…", "argument requires that…", "returning this value requires that…")

The labels used in this chapter, and what each one marks:

Label	Marks
`borrowed value does not live long enough`	a borrow whose region must reach past the end of its value
`binding `x` declared here`	where the borrowed variable was created
`` `x` dropped here while still borrowed ``	the end of the value's scope, inside the region
`... borrow occurs here`	the start of a borrow involved in a conflict
`... later used here`	the use that keeps a borrow live across a conflict
`type annotation requires that ...`	a written type that imposes a region

Every fix removes one of the three ingredients of the timeline. For the dangling caller, all of these work:

- **Use the result earlier**, while the borrowed value still exists: move the `println!` into the inner block (the `use_inside` caller above).
- **Make the borrowed value live longer**: declare `inner` in the outer block, next to `outer`.
- **Stop borrowing**: make the result own its data, so that it no longer needs the inputs. The second caller in `code/examples/ch01_longest.rs` does this:

```
fn own_result() {
    let outer = String::from("a longer string");
    let result: String;
    {
        let inner = String::from("short");
        result = longest(outer.as_str(), inner.as_str()).to_owned(); // a copy
    }
    println!("{result}"); // fine: `result` borrows nothing
}
```

- **Correct the contract**, when the signature promises less than the function could: that was the fix for `first`, whose result never borrows from `y`.

Which fix is right depends on what the program means; the diagnostic only tells you that the current combination is contradictory. Chapter 14 discusses the design side of the choice (own or borrow).

1.8 Notation

The rest of the book uses a small vocabulary of lifetime syntax. Here it is in one place; the last column says where each form is explained in depth.

Syntax	Read as	Meaning	More in
`'a`	"lifetime a"	a named region, a parameter	this chapter
`&'a T`, `&'a mut T`	"a reference with lifetime `'a`"	valid throughout `'a`	this chapter
`'a: 'b`	"`'a` outlives `'b`"	`'a` contains every point of `'b`	Chapter 4
`T: 'a`	"`T` outlives `'a`"	every borrow inside `T` outlives `'a`	Chapter 7
`'static`	"static"	the region of the whole program run	Chapter 7
`'_`	"anonymous lifetime"	a lifetime the compiler picks by position	Chapter 6
`for<'a>`	"for all `'a`"	a bound that holds for every lifetime	Chapter 9
`use<'a, T>`	"uses `'a` and `T`"	what an `impl Trait` may capture	Chapter 8

Every form in the table appears in `code/examples/ch01_notation.rs`, which compiles; each function is a minimal use of one form:

```
// `'a` names a region; `&'a T` is a shared reference valid throughout it.
fn first_line<'a>(text: &'a str) -> &'a str {
    text.lines().next().unwrap_or("")
}

// `&'a mut T`: an exclusive reference, valid throughout `'a`.
fn append<'a>(buffer: &'a mut String, word: &str) -> &'a mut String {
    buffer.push_str(word);
    buffer
}

// `'long: 'short` ("'long outlives 'short"): every point of `'short` is in `'long`.
fn either<'short, 'long: 'short>(a: &'short str, b: &'long str) -> &'short str {
    if a.is_empty() { b } else { a }
}

// `T: 'a`: every borrow inside `T` is valid throughout `'a` (Chapters 7 and 8).
fn boxed<'a, T: Display + 'a>(value: T) -> Box<dyn Display + 'a> {
    Box::new(value)
}

// `'static`: the region that covers the whole run of the program (Chapter 7).
const GREETING: &'static str = "hello";

// `'_`: "some lifetime", here the one of `text` (Chapter 6).
fn words(text: &str) -> std::str::SplitWhitespace<'_> {
    text.split_whitespace()
}

// `for<'a>`: `F` must accept a borrow of every lifetime `'a` (Chapter 9).
fn apply<F>(f: F) -> usize
where
    F: for<'a> Fn(&'a str) -> &'a str,
{
    let local = String::from("built inside apply");
    f(&local).len()
}

// `use<'a>`: the lifetimes an `impl Trait` return type may capture (Chapter 8).
fn lengths<'a>(text: &'a str) -> impl Iterator<Item = usize> + use<'a> {
    text.split_whitespace().map(str::len)
}
```

```
regions, not durations
lifetimes are regions
hello
regions
5
18
[8, 3, 9, 6, 4]
```

Two of these deserve a word now. The **outlives** relation `'long: 'short` says that the region `'long` contains every point of `'short`. "Outlives" is meant inclusively: every lifetime outlives itself, so `'a: 'a` always holds. A reference valid for a larger region can always be used where a smaller one is expected, which is why `either` can return `b`, a `&'long str`, as a `&'short str`. Chapter 4 shows that this is a form of subtyping, and what happens to it inside other types.

The bound `T: 'a` applies the same idea to a type: every reference (and every lifetime parameter) inside `T` outlives `'a`. A type that contains no borrows at all, such as `String` or `u64`, outlives every lifetime, including `'static`. That is why `boxed` can be called with a `&str` borrowed from `text`: `T` is

`&'x str` for the region `'x` of that borrow, and the bound `T: 'a` holds for any `'a` inside `'x`. Chapter 7 is about the most common such bound, `T: 'static`, and why it does not mean "lives forever".

Implied bounds

Some outlives relations never need to be written, because the types in a signature already imply them. A reference `&'a T` is only meaningful if what it points to is valid for all of `'a`, so the type `&'a T` is **well formed** only if `T: 'a`. The compiler assumes that the types in a function's signature (its parameters and its return type) are well formed, and therefore that every relation they imply holds. These are called **implied bounds** (`code/examples/ch01_implied_bounds.rs`):

```
// `&'a &'b str` is a valid type only if `'b: 'a`, so the body may rely on it.
fn inner<'a, 'b>(outer: &'a &'b str) -> &'a str {
    *outer // a `&'b str`, returned as a `&'a str`
}

// No parameter type implies `'b: 'a` here, so the signature must say it.
fn pick<'a, 'b>(x: &'a str, y: &'b str, first: bool) -> &'a str
where
    'b: 'a,
{
    if first { x } else { y }
}

// A field `&'a T` implies `T: 'a`: the struct need not write it.
struct Labeled<'a, T> {
    label: &'a str,
    value: &'a T,
}

fn value_of<'a, T>(labeled: &Labeled<'a, T>) -> &'a T {
    labeled.value
}
```

```
config.toml
config.toml
port = 8080
```

- `inner` receives a `&'a &'b str`: a reference, valid for `'a`, to a reference valid for `'b`. That type is well formed only if `&'b str: 'a`, that is, if `'b: 'a`. So the body may return the inner `&'b str` as a `&'a str`, without any `where` clause.
- `pick` has the same body as the rejected `longest` above, but its parameters are two separate references, and nothing in their types relates `'a` and `'b`. The relation must be written, and the `where 'b: 'a` clause is exactly the bound that the `help` line suggested. Now the body compiles, and every caller must pass a `y` that is valid at least as long as `x`'s region.
- `Labeled<'a, T>` has a field of type `&'a T`, which implies `T: 'a`. The struct does not have to say so, and neither does `value_of`, whose parameter type mentions `Labeled<'a, T>`. Older code often writes `struct Labeled<'a, T: 'a>`; the bound has been inferred since Rust 1.31, as the release announcement linked above puts it: "The `: 'a` is inferred."

Implied bounds are why you rarely write `T: 'a` by hand. When you do need it, it is because a type you produce (a boxed trait object, an `impl Trait`, a thread's closure) must carry a borrow that no parameter type already constrains, and the compiler asks for the bound explicitly (Chapters 7 and 8).

1.9 Common confusions

"**A lifetime is how long a value lives.**" A lifetime is the region where a *reference* may be used. The value it points to usually lives much longer. The scope of the value is an upper bound on the region, not the region itself.

"**The borrow lasts until the closing brace.**" Not since NLL. A borrow ends after the last use of the reference or of anything derived from it, path by path. One exception you will meet in Chapter 5: a value whose type implements `Drop` is used once more when it is dropped at the end of its scope, and a borrow it holds is live until then.

"**Adding `'a` or `'static` can fix a 'does not live long enough' error.**" An annotation adds a requirement; it never extends a value's scope. The `&'static str` annotation above *caused* the error. Lifetime errors are fixed by changing where values live, where references are used, who owns the data, or which relationship the signature claims.

"**`'a` in `longest` is the lifetime of `x`.**" It is a region that the caller picks at each call, subject to both arguments being valid throughout it. With arguments of different lengths of validity, `'a` can be at most the part where both are valid.

"**If the body only returns `x`, callers can rely on that.**" Callers see only the signature. To let them rely on it, say it in the signature (`ch01_signature_fixed.rs`).

"**Lifetimes are checked at run time, or cost something at run time.**" They are erased before code generation. A program is either proven safe at compile time or rejected.

"**Two mutable borrows of the same value can never coexist in a function.**" Their *regions* must not overlap. Two mutable borrows one after the other, each ending before the next begins, are fine (`ch01_aliasing.rs`).

1.10 Summary

- For the borrow checker, a lifetime is a **region**: a set of points in the control-flow graph of a function body. It has no run-time representation; it is computed, checked and erased before code generation.
- The scope of a value, the scope of a variable holding a reference, and the region of a borrow are three different things. The region is the smallest, and must fit inside the scope of the borrowed value.
- With **non-lexical lifetimes**, a borrow is live from its creation to the last use of the reference or of anything derived from it, along each path. A borrow can be live on one branch and dead on another, and live across a loop's back edge.
- **Aliasing XOR mutation**: at every point, a value has any number of shared borrows or one mutable borrow. Overlapping regions produce E0502 (shared and mutable), E0499 (two mutable) and E0506 (assignment while borrowed).

- Lifetime annotations **describe relationships**; they never extend anything. In `longest`, one name `'a` means that the result is usable only where both inputs are valid. E0597 says that a borrow would have to outlive its value.
- **The signature is the contract**: callers are checked against the signature only (E0597 at the call), and the body is checked against it ("lifetime may not live long enough" in the body).
- Read a diagnostic as a timeline: where the borrow starts, what conflicts with it or ends its value, which later use keeps it alive, and where a requirement comes from. Fix it by moving a use, moving a value, owning the data, or correcting the contract.
- Notation: `'a`, `&'a T`, `'a: 'b`, `T: 'a`, `'static`, `'_`, `for<'a>`, `use<..>`. Implied bounds: a type such as `&'a T` implies `T: 'a`, so such bounds are rarely written by hand.

1.11 Exercises

1. In `code/examples/ch01_nll.rs`, add the line `let last = &v[2];` before the `push`, and use `last` once before the `push` and once after it. Predict the error code and the three labels before compiling. Then remove the use after the `push` and check that the program compiles again. *Hint:* only the use after the `push` puts the `push` inside the region of the new borrow.
2. Change `describe_or_extend` in `code/examples/ch01_branches.rs` so that the `then` branch prints `first` after its `push`. Explain, in terms of the paths of the control-flow graph, why the error points at the `push` and not at the `if`. Then move the `println!` before the `push` and check that the function compiles again. *Hint:* on the `then` path, the new use comes after the `push`, so the borrow is now live at the `push` on that path.
3. Write a function `fn starts_with_either<'a>(text: &'a str, a: &str, b: &str) -> Option<&'a str>` that returns `text` if it starts with `a` or `b`, and call it with `a` and `b` borrowed from strings that are dropped before the result is used. Then change the signature to use `'a` for all three parameters and explain the new error. *Hint:* in the first version, the elided lifetimes of `a` and `b` are independent of `'a` (Chapter 3 explains why), so the result does not depend on them.
4. Fix `code/compile_fail/ch01_longest_dangling.rs` in three different ways: by moving the use, by moving a declaration, and by owning the result. For each fix, say which of the three labels of the original diagnostic it makes disappear. *Hint:* `code/examples/ch01_longest.rs` shows two of the fixes; the third moves `let inner` next to `let outer`.
5. Write `fn first_of<'a, 'b>(items: &'a [&'b str]) -> &'a str` returning the first element (or `""`). Does it need a `where` clause? Now change the parameter type to two separate parameters, `first: &'b str, _rest: &'a [u8]`, keep the return type `&'a str`, and see what the compiler suggests. *Hint:* `&'a [&'b str]` is well formed only if `'b: 'a`; that is an implied bound. With two separate parameters, nothing implies it.
6. Explain why `std::any::TypeId::of::<T>()` requires `T: 'static`, using what this chapter says about erasure. Then check your explanation against Chapter 7. *Hint:* if `&'a str` and `&'static str` had different `TypeId`s, a run-time value would depend on a lifetime that no longer exists when the program runs.